

Anchor Parsing

May 2026

Abstract We present a parser that given a context free grammar $G = (N, \Sigma, P, S)$ and a string β such that $\exists x, y \in \Sigma^* : x\beta y \in L(G)$, enumerates all possible parse trees that can cover β with the missing left and right context, if any. We can use this to extract parse trees from incomplete code, such as fragments found in git patches.

1 Introduction

Git patches only record the added and removed lines in a codebase, presenting the code as fragments with minimal surrounding context. When studying how code changes across a series of patches, there is a need to understand what the changed code is syntactically: a declaration, a function call, a statement, etc.

```
int size, rc = 0;

while (n > 0) {
- size = zpci_get_max_write_size((u64 __force)
src,
- (u64) dst, n,
- ZPCI_MAX_READ_SIZE);
+ size = zpci_get_max_io_size((u64 __force) src,
+ (u64) dst, n,
+ ZPCI_MAX_READ_SIZE);
rc = zpci_read_single(dst, src, size);
if (rc)
break;
```

A fragment from a Linux kernel patch.

```
int size, rc = 0;

while (n > 0) {
size = zpci_get_max_io_size((u64 __force) src,
(u64) dst, n,
ZPCI_MAX_READ_SIZE);
rc = zpci_read_single(dst, src, size);
if (rc)
break;
```

Code to be parsed (highlighted) with surrounding context.

Parsing this input is complicated, since standard parsers assume a well-formed, complete input. When presented with a fragment they fail at the first token that cannot extend a valid parse due to the missing surrounding context. Parsers with error recovery strategies (tree-sitter and similar tools) salvage a single best parse but suppress ambiguity that might exist.

To resolve these issues, we change the basic assumptions.

1. We assume that the input string belongs to the language and has a valid derivation
2. We only need the smallest non terminals that can cover the entire input.

Given a context-free grammar G and a token sequence β , our algorithm returns the covering set $\mathcal{C}(\beta) \subseteq N$ of nonterminals under which β appears as a substring, together with parse trees rooted at nonterminals in $\mathcal{C}(\beta)$ and descriptions of the missing context. This allows us to compare two fragments with the same covering set directly as instances of the same grammatical construct.

In this paper we describe an adaptation of the h-cover framework of Satta and Stock [1] to the fragment parsing problem, returning $\mathcal{C}(\beta)$ with parse trees and context descriptions for any context-free grammar. We evaluate it on real fragments drawn from Linux kernel patches.

2 Problem

Let $G = (N, \Sigma, P, S)$ be a context-free grammar and $\beta \in \Sigma^+$ a token sequence. A nonterminal $A \in N$ covers β if there exist $\alpha, \gamma \in \Sigma^*$ such that

$$A \xRightarrow{*} \alpha\beta\gamma$$

The *covering set* is

$$\mathcal{C}(\beta) = \{A \in N \mid \exists \alpha, \gamma \in \Sigma^* : A \xRightarrow{*} \alpha\beta\gamma\}$$

The algorithm computes, for each $A \in \mathcal{C}(\beta)$, a set of *gap parse trees* rooted at A spanning β . A gap parse tree is a parse tree over β whose leaves are either tokens from β or *virtual nodes* $\langle X \rangle$ ($X \in N \cup \Sigma$). Virtual nodes mark constituents present in the derivation of A but absent from β . Those appearing before the first token form the left gap L ; those after the last token form the right gap R .

Example. Let G_{cl} be the grammar [1]:

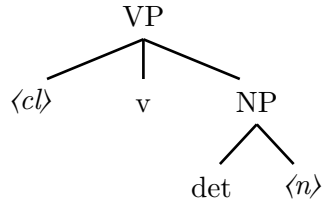
$$\begin{aligned} S &\rightarrow NP \ VP \quad (\text{head: VP}) \\ VP &\rightarrow cl \ v \ NP \quad (\text{head: v}) \\ NP &\rightarrow det \ n \quad (\text{head: det}) \end{aligned}$$

For the fragment

$$\beta = v \ det$$

VP derives $cl \cdot \beta \cdot n$, placing it in the covering set. S derives $NP \ cl \cdot \beta \cdot n$ similarly, giving $\mathcal{C}(\beta) = \{VP, S\}$.

The gap parse tree for VP (left gap $\langle cl \rangle$, right gap $\langle n \rangle$) is:



$\langle cl \rangle$ is the missing left sibling and $\langle n \rangle$ the missing right terminal, giving $L_{VP} = (\langle cl \rangle)$ and $R_{VP} = (\langle n \rangle)$. Boundary seeding promotes VP into S, prepending $\langle NP \rangle$, so $L_S = (\langle NP \rangle, \langle cl \rangle)$ and $R_S = (\langle n \rangle)$.

3 Terminology

The **recognition table** T is an $(n+1) \times (n+1)$ array for an input of length n . Cell $T[i, j]$ holds items representing derivations assembled over the span $w_{i+1} \dots w_j$.

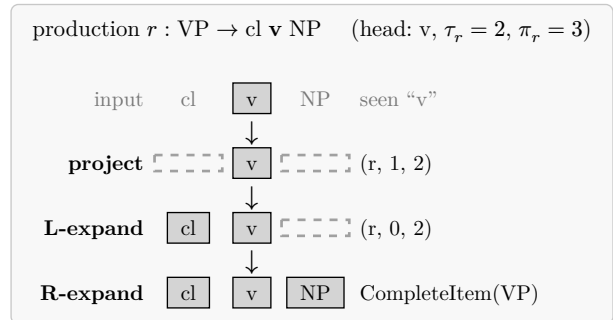
Two kinds of item appear in T :

- A **complete item** A which asserts that nonterminal A derives the span exactly.
- A **partial item** (r, s, t) that represents partial progress on production $r : D \rightarrow Z_1 \dots Z_{\pi_r}$ with head at position τ_r .

Each production $r : D \rightarrow Z_1 \dots Z_{\pi_r}$ in the grammar designates one symbol as its **head**, at position τ_r for $1 \leq \tau_r \leq \pi_r$.

A partial item starts from the head symbol Z_{τ_r} , seeded by **projection**, and grows outward through **left** and **right expansion**. The head always stays within the assembled range ($s < \tau_r \leq t$). When $s = 0$ and $t = \pi_r$ the full right-hand side is covered.

For example, in the production $VP \rightarrow cl \ v \ NP$, we assign v as the head. Seeing the token “v” in the input projects it to the partial item $(r_{VP}, 1, 2)$. Then we search for “cl” and NP to complete it.



Projection is the step that creates an initial partial item from a recognized head symbol, without consuming additional input. For a unit production ($\pi_r = 1$), the head projects directly to a complete item.

Expansion is the step that extends a partial item by combining it with an adjacent item in the table. A **left expansion** consumes the next required symbol to the left while a **right expansion** does the same to the right. For non-unit productions, a complete item is produced only by the final expansion step, not by projection.

The **h-cover** $\mathcal{H}(G)$ is a set of production rules derived from the grammar. It is essentially an intermediate grammar that preserves the original language. It is computed once and can be reused across all inputs.

Virtual nodes $\langle X \rangle$ represent constituents that participate in a derivation but are absent from the input fragment. They give us possible completions of the fragment.

The **worklist** is the queue that drives recognition. Items are added to the worklist when first placed into any cell and processed at most once per cell. Processing an item triggers projection and expansion steps that may add further items.

L-Reduce and **R-Reduce** are passes applied when the full span $T[0, n]$ is empty after the worklist empties. L-Reduce processes prefix spans $T[0, k]$ by injecting virtual left siblings for items whose left context is missing from the fragment. R-Reduce does the same for suffix spans $T[k, n]$ when L-Reduce is insufficient.

4 Overview of the algorithm

Fix a head position in each production of G and precompute the h-cover $\mathcal{H}(G)$ [1]. This runs once per grammar and is shared across all inputs.

Initially, each token w_i adds projected items into $T[i - 1, i]$, yielding initial partial items or complete items. These items are then added to the worklist and then processed. Items in

$T[i, j]$ extend left by combining with any item in $T[i', i](i' < i)$ that provide the required symbol, placing the result in $T[i', j]$. For right expansions, items combine with any item in $T[j, j'](j' > j)$, placing the result in $T[i, j']$.

A fragment missing its surrounding context cannot be assembled to $T[0, n]$ from tokens alone. Before the worklist runs, the edge cells $T[0, 1]$ and $T[n - 1, n]$ are seeded with items whose missing left or right sibling is inferred from the grammar and recorded as a virtual node. The inferred items combine with the boundary tokens to produce items at the fragment boundary. This allows for enough context to participate in normal expansion.

If $T[0, n]$ is still empty after the worklist is empty, L-Reduce processes each prefix span $T[0, k](k < n)$, climbing the h-cover from items already there. It treats the required left sibling at each step as virtual, and adds the resulting items back to $T[0, k]$. The worklist then runs on those new items, which can right-expand toward $T[0, n]$. If $T[0, n]$ is still empty, R-Reduce does the same from each suffix span $T[k, n]$, treating the right sibling as virtual and left-expanding toward $T[0, n]$.

Boundary seeding

$$\boxed{\langle ? \rangle} + \boxed{w_1} \rightarrow \boxed{T[0, 1]}$$

$$\boxed{w_n} + \boxed{\langle ? \rangle} \rightarrow \boxed{T[n - 1, n]}$$

virtual siblings paired with edge tokens, seeded into edge cells

L-Reduce

$$\boxed{\langle ? \rangle} + \mathcal{C}(\beta_k) \rightarrow T[0, k]'$$

new items in $T[0, k]'$ right-expand toward $T[0, n]$, populating $\mathcal{C}(\beta)$

R-Reduce

$$\mathcal{C}(\beta_{-k}) + \boxed{\boxed{?}} \rightarrow T[k, n]'$$

new items in $T[k, n]'$ left-expand toward $T[0, n]$, populating $\mathcal{C}(\beta)$

Complete items in $T[0, n]$ are the covering nonterminals. The parse tree for each is reconstructed from derivation pointers stored during recognition; virtual nodes in the tree give the left and right gap descriptions directly.

5 Algorithm

5.1 Recognition table

For input $\beta = w_1 \dots w_n$, items in $T[i, j]$ cover the span $w_{i+1} \dots w_j$.

5.2 H-Cover

The h-cover $\mathcal{H}(G)$ encodes three families of inference rule derived from the grammar, computed once at preparation time and reused across all inputs.

For each production $r : D \rightarrow Z_1 \dots Z_{\pi_r}$ with head at position τ_r :

Projections : The head symbol Z_{τ_r} seeds an initial partial item $(r, \tau_r - 1, \tau_r)$ spanning only the head position. For unit productions ($\pi_r = 1$), the head projects directly to a complete item for D . **Left expansions :** A partial item (r, s, t) with $s > 0$ requires Z_s as its immediate left neighbour. If Z_s is assembled over $T[i', i]$, the item combines with it to yield $(r, s - 1, t)$ in $T[i', j]$.

Right expansions : Symmetrically, a partial item (r, s, t) with $t < \pi_r$ requires Z_{t+1} as its immediate right neighbour. If Z_{t+1} is assembled over $T[j, j']$, it yields $(r, s, t + 1)$ in $T[i, j']$.

Epsilon-nullable nonterminals admit a further class of **epsilon projections**: if an expansion step expects a nonterminal B that derives ε , the

partial item may advance without finding B in the table.

5.3 Worklist

Items are enqueued when first added to a cell; duplicates are discarded. Dequeueing item a from $T[i, j]$ triggers four operations:

1. **Project.** If a is a complete item and $(i, j) \neq (0, n)$, find each production where its nonterminal serves as the head symbol and add the initial partial item for that production to $T[i, j]$ (a complete item directly, for unit productions).
2. **Left-expand.** If a is a partial item (r, s, t) with $s > 0$, probe all cells $T[i', i]$ for Z_s and add $(r, s - 1, t)$ to $T[i', j]$ for each match.
3. **Right-expand.** If a has $t < \pi_r$, probe all cells $T[j, j']$ for Z_{t+1} and add $(r, s, t + 1)$ to $T[i, j']$ for each match.
4. **Reverse.** Item a may serve as a left or right child in an expansion assembled in another cell. For each production in the h-cover where a can play this role, the table is probed for the complementary sibling and the combined item is enqueued.

Because each item is enqueued at most once and the table is finite, the worklist terminates.

5.4 Boundary seeding

When β is a proper infix — its first token needs a left sibling or its last needs a right sibling absent from the input — the worklist cannot assemble items over $T[0, n]$ from tokens alone.

Initial seeding. Before the worklist runs, the edge cells $T[0, 1]$ and $T[n - 1, n]$ receive items whose derivation requires a missing neighbour. For any item derivable from w_1 that requires a left sibling absent from β , we inject it with a virtual node recording the missing constituent; the symmetric injection applies at $T[n - 1, n]$. These virtual nodes become leaves in the reconstructed parse tree.

L-Reduce. If $T[0, n]$ is empty after the worklist empties, we process each prefix span $T[0, k]$ for $k = 0, \dots, n - 1$: items at $T[0, k]$ that can extend leftward with a virtual left sibling are injected and the worklist re-run.

R-Reduce. If $T[0, n]$ is still empty, the same process runs on suffix spans $T[k, n]$ for decreasing k . A final closure pass then runs on $T[0, n]$ directly, combining prefix and suffix derivations that meet at the full span.

5.5 Root extraction

Every complete item A in $T[0, n]$ identifies a nonterminal covering β . The covering set $\mathcal{C}(\beta)$ is assembled from these; for each, the parse tree is reconstructed by following derivation pointers stored during recognition.

A **subtree-dominance filter** removes redundant candidates: if the best parse tree for A appears as a direct subtree of the best parse tree for B , then A is dominated by B and excluded. Surviving candidates are ranked by gap count so the most complete interpretations appear first.

6 Pseudocode

Recognition takes a prepared grammar and an input fragment β . The table T is initialised empty. Three seeding steps populate the first cells: epsilon projections for nullable nonterminals, initial partial items for each input token at its span, and boundary items at the edges $T[0, 1]$ and $T[n - 1, n]$ for fragments missing left or right context. The worklist then runs to saturation.

If $T[0, n]$ is still empty after the worklist empties — meaning no item spans the full fragment — L-Reduce fires: for each prefix span $T[0, k]$, items there are extended with a virtual left sibling and the worklist re-runs. If $T[0, n]$ remains empty, R-Reduce does the same from suffix

spans. A final frontier pass then runs directly on $T[0, n]$ to promote any items that arrived via inductive fill.

```

recognize(pg,  $\beta = w_1 \dots w_n$ )
  initialise  $(n + 1) \times (n + 1)$  table  $T$ 
  seed epsilons; seed terminals into  $T[i - 1, i]$  for
    each  $w_i$ 
  seed left boundary  $T[0, 1]$ ; seed right boundary
     $T[n - 1, n]$ 
  process_agenda( $T$ )
    if  $T[0, n] = \emptyset$ :
      for  $k = 1$  to  $n$ : L-reduce at  $T[0, k]$ ;
        process_agenda( $T$ )
      if  $T[0, n] = \emptyset$ :
        for  $k = n - 1$  downto  $0$ : R-reduce at
           $T[k, n]$ ; process_agenda( $T$ )
    closure      pass      on       $T[0, n]$ ;
    process_agenda( $T$ )
  frontier      pass      on      new  $T[0, n]$  items;
  process_agenda( $T$ )
  return  $T$ 

```

Figure 1: Recognition

Each dequeued item triggers up to four operations. A complete item projects into any production where its nonterminal is the head, seeding a new partial item in the same cell. A partial item (r, s, t) with $s > 0$ probes every cell $T[i', i]$ to its left for Z_s ; each match yields $(r, s - 1, t)$ in $T[i', j]$. Symmetrically, $t < \pi_r$ triggers a rightward probe. The reverse step handles the dual case: item a may be the sibling that some existing partial item was waiting for, so the table is probed for those blocked items and the combinations are enqueued. An item is enqueued at most once, so the worklist terminates.

```

process_agenda( $T$ )
  while worklist  $\neq \emptyset$ :
    dequeue  $(a, i, j)$ 
    if  $a$  is a complete item and  $(i, j) \neq (0, n)$ :
      project into productions where  $a$  is head
      if  $a = (r, s, t)$  with  $s > 0$ : left-expand with
         $T[i', i]$  for all  $i' \leq i$ 
      if  $a = (r, s, t)$  with  $t < \pi_r$ : right-expand
        with  $T[j, j']$  for all  $j' \geq j$ 
      reverse: probe for partial items that can use
         $a$  as a child

```

Figure 2: Worklist

7 Implementation

The implementation is in OCaml and separates grammar preparation from recognition, so that the same prepared grammar is reused across an entire corpus of fragments.

Grammar pipeline. Grammars are read from ANTLR4 `.g4` files. The reader strips comments, splits rules, and desugars operator notation: $x+$ becomes $x \ x^*$, and x^* introduces a fresh nonterminal with productions $x^* \rightarrow x \ x^* \mid \varepsilon$. Uppercase identifiers and single-quoted strings become terminals; lowercase identifiers become nonterminals. The pipeline produces a flat record of nonterminals, terminals, productions, and a designated start symbol.

Preparation. `prepare(G)` computes the h-cover together with two input-independent auxiliary tables. The *nullable set* identifies nonterminals deriving ε and drives epsilon projections. The *min-yield table* maps each nonterminal to the shortest terminal string it derives; virtual nodes in the output are labelled with this completion rather than an abstract nonterminal name.

Recognition. `recognize(H, beta)` seeds epsilons and boundary items into the empty table, runs the worklist, and applies L-Reduce and R-Reduce as described above. Derivation pointers are stored alongside each item so that parse

trees can be reconstructed after recognition completes.

Tree reconstruction. Trees are reconstructed lazily from the stored derivation pointers. Reconstruction is capped at five trees per root to avoid cartesian-product blowup in ambiguous grammars; in practice the first tree suffices to identify the syntactic category and gap description.

8 Evaluation

We evaluate on three grammars of increasing complexity. Grammar preparation runs once per grammar; recognition and reconstruction run per fragment.

GCL grammar. Three productions, 4 terminals. For the fragment $\beta = v \text{ det}$ the system returns one root: VP with 2 gaps ($L = \langle \text{cl} \rangle$, $R = \langle \text{n} \rangle$) and a single parse tree. S is inferred but removed by the subtree-dominance filter, since S’s best tree contains VP as a direct child — retaining both would be redundant. For the complete input “det n cl v det n” the system returns S with 0 gaps and a single tree.

Lisp grammar. An S-expression grammar read from an ANTLR4 file, with alternation and Kleene operators desugared by the grammar pipeline. Single-atom inputs are covered by `s_expression` with 0 gaps. Dotted pairs such as `(ATOM . ATOM)` are likewise covered with 0 gaps.

C grammar. A full C grammar in ANTLR4 format, desugared to several hundred productions. Grammar preparation completes in under one second and is shared across all fragments. For short fragments of 2–10 tokens, recognition produces tens to hundreds of table items. Multiple root candidates appear naturally: a bare sequence of tokens may be a valid expression, a statement argument, a declarator, or several simultaneously. The subtree-dominance filter reduces the displayed roots to non-redundant

candidates. Tree reconstruction is lazy and capped at 5 trees per root to avoid cartesian-product blowup in the derivation forest; this suffices to identify the syntactic category and gap description for each root.

9 Related Work

Substring recognition. Rekers and Koorn [2] give an $O(n^3)$ algorithm for deciding whether β is a substring of some sentence in $L(G)$, and produce parse trees. Their trees are for the complete sentence $\alpha\beta\gamma$ rooted at S , not for β alone rooted at covering nonterminals. The covering set $\mathcal{C}(\beta)$ is not computed.

Osorio and Navarro [3] give the closest prior formulation: using a CYK variant they compute $\{A \in N \mid \beta \in L^{\text{infix}(A)}\}$ explicitly. No parse trees are produced, no context description is given, and inputs that are not clean infixes of any sentence are not handled.

Gap parsing. Lang [4] parses strings with explicit gap markers (? for one unknown word, * for a sequence). The parser labels gap positions with covering nonterminals. Our setting differs: we have no markers and no knowledge of gap positions — the entire surrounding context is unknown.

Bidirectional parsing. Satta and Stock [1] develop the h-cover framework for bidirectional tabular recognition of complete input. We adopt the framework but change the goal: instead of recognising membership in $L(G)$, we classify a bare fragment by computing $\mathcal{C}(\beta)$ with parse trees and context descriptions. Partial items, intermediate in the original algorithm, become the primary output of ours.

Error recovery. Tree-sitter and similar tools find a single best repair when parsing fails. This is appropriate for editors but not for patch analysis, where multiple grammatical classifica-

tions may simultaneously hold for the same fragment.

10 Conclusions

We have defined the fragment parsing problem and described an algorithm for solving it: given any context-free grammar and a bare token sequence, compute the covering set $\mathcal{C}(\beta)$ of nonterminals whose infix language contains β , together with parse trees and context descriptions for each. The algorithm adapts the h-cover framework of Satta and Stock to operate without a complete input or a privileged start symbol. We evaluate it on C fragments from Linux kernel patches.

Limitations. Tree reconstruction is expensive for long fragments against large grammars; lazy reconstruction is future work. The algorithm assumes the fragment has a valid derivation — genuinely malformed inputs are not currently flagged as such.

Future work. Probabilistic ranking of root candidates; application to additional languages via existing ANTLR grammars; a study of syntactic change patterns in Linux kernel patches using $\mathcal{C}(\beta)$ as the classification signal.

Bibliography

- [1] G. Satta and O. Stock, “BiDirectional Context-Free Grammar Parsing for Natural Language Processing,” *Artificial Intelligence*, vol. 69, pp. 123–164, 1994.
- [2] J. Rekers and W. Koorn, “Substring Parsing for Arbitrary Context-Free Grammars,” in *Proceedings of the Second International Workshop on Parsing Technologies (IWPT)*, 1991, pp. 218–224.
- [3] M. Osorio and J. A. Navarro, “Decision Problem of Substrings in Context Free Languages,” in *Proceedings of CIC 2001*, 2001.
- [4] B. Lang, “Parsing Incomplete Sentences,” in *Proceedings of the 12th International Conference on Computational Linguistics (COLING)*, 1988, pp. 365–371.